

INF-203 Project Report

Knowledge Graph System

Course: INF-203

Izza Qamar
Martin Braaten Kildal
Philip Adrian Kern Øvland

Norwegian University of Life Sciences

June 2026

1 Task 1: Group collaboration logistics

Our group created a shared GitHub repository for the project. The repository is used to store code and project files, allowing all members to contribute and track changes. This setup helps document each member's contributions and supports collaborative development throughout the course. The repository can be accessed here: [Inf-203 GitHub repository](#). For informal communication we used a whatsapp group chat to keep contact between classes.

2 Task 2: Requirements management

We created a requirements register using a spreadsheet to manage the system requirements. The requirements are structured using an Agile approach with epics and user stories, and are prioritised using the MoSCoW method. The register is maintained and refined iteratively throughout the project.

3 Task 3: Incidence-list labelled graph data structure

For this task, we first set up three basic classes: Node, Edge, and Graph. Then we made each node store its own unique label along with lists of incoming and outgoing edges. After that, we created the Edge class so every edge keeps its label and knows which nodes it connects. Finally, we built the Graph class to manage everything, it keeps track of all nodes, makes sure labels are unique, and links edges into the right incoming/outgoing lists. Together, this gives us doubly linked incidence list graph that works for our knowledge-graph data.

4 Task 4: Graph data input from JSON-LD

We implemented a `JsonLD_parser` class with a `load_jsonld` method that reads a JSON-LD file, recursively extracts triples (subject, predicate, object), and builds a graph from them. The parser handles both the no-context and context forms of JSON-LD, satisfying both the minimum requirement and the extended goal. It correctly handles nested objects, lists, typed literals, and skips `@context` metadata to prevent false triples.

We ran a check run on the file: `linked-data-intro-context.json`

5 Task 5: Encapsulation and docstrings

We extended the graph implementation by adding property methods (getters and setters) for the main class attributes. The Node class was updated with properties for labels, incoming edges, outgoing edges, and attributes. The Edge class received properties for labels, source nodes, and target nodes, while the Graph class provides controlled access to its collections of nodes and edges.

After implementing the properties, we created a series of test cases to verify that nodes and edges were added correctly and that incoming and outgoing incidence lists were maintained as expected. The tests confirmed that each node stores the correct relationships and that edges are linked to both their source and target nodes. This verified that the graph structure functions correctly and remains suitable for representing knowledge-graph data.

6 Task 6: Linear queries for paths in the knowledge graph

The objective of this task was to implement support for linear path queries in the knowledge graph. A linear path query is defined as an ordered list of edge labels, and the system returns

all node pairs connected by a path that follows these labels in sequence.

We implemented this through a separate class `Query`, which handles path query-related functionality. The method `query_path_forward(labels)` is defined inside this class. The method starts by considering all nodes in the graph as potential starting nodes. For each label in the query, it follows outgoing edges that match the required label (input label) and extends the current paths. After we process all labels, the remaining paths represent valid matches for the query. The method then outputs the start node (`x0`) and the final node reached after following the complete sequence of labels.

For testing, we executed:

```
labels = [  
    "https://w3id.org/dppo/ontology/containsInformation",  
    "https://w3id.org/dppo/ontology/hasPart",  
]
```

```
query.query_path_forward(labels)
```

This query searches for all paths where a node is connected through the property 'containsInformation' to an intermediate node, which is then connected through the property 'hasPart' to a final node.

We loaded the instance data (IndividualWindowDPP) provided in Turtle (TTL) format into Protégé and converted to JSON-LD. The JSON-LD files were successfully imported into the program and we used it to construct the knowledge graph. To validate the implementation, we executed equivalent SPARQL queries in Protégé.

The equivalent SPARQL query executed in Protégé was:

```
PREFIX dppo: <https://w3id.org/dppo/ontology/>  
  
SELECT ?x0 ?x2  
  
WHERE {  
    ?x0 dppo:containsInformation ?x1 .  
    ?x1 dppo:hasPart ?x2 .  
}
```

We compared it with the output of `Query.query_path_forward()`. The same start and end node pairs were returned in both cases, which confirms that the implementation correctly evaluates linear path queries.

Thus the system can load the provided RDF data, execute forward linear path queries over the knowledge graph, and produce results consistent with SPARQL query evaluation in Protégé.

7 Task 7:

We have updated our "User stories" based on our needs for the code and what we see as mandatory functions in our code. We added multiple general user stories and one each, based on what we have worked on this far. These will from now on be our own responsibilities to follow up upon, and to keep them updated. We also added a column for whose responsibility a user story is (name), if it is implemented (Y/N) and if it is tested (Y/N).

8 Task 8: Advanced OOP techniques

We used advanced OOP by passing callable filter objects as arguments in the `Filter` class

The filters (`FilterByType`, `FilterByName`, `FilterOrphans`) implement `__call__`, allowing them to be used like functions. This lets us filter nodes during traversal without changing the main query logic.

This makes the system more flexible and reusable because different filtering behaviors can be added.

9 Task 9: Generalizing the querying and analysis functionalities

We implemented this by adding the `query_path` method which accepts `directions` list alongside the `labels` list:

```
def query_path(self, labels, directions):
```

Each entry in `directions` is a boolean where `True` means follow the edge forward and `False` means follow it backward. This allows us to run mixed-direction queries such as the backward traversal used in our `check_W2`:

```
query.query_path(  
    labels=["https://w3id.org/dppo/ontology/hasPart"],  
    directions=[False]  
)
```

During traversal, the method selects outgoing or incoming edges depending on the direction:

```
edges = last.outgoing if forward else last.incoming
```

For each matching edge, the next node is chosen based on whether we are moving forward or backward:

```
next_node = edge.target if forward else edge.source  
new_paths.append(path + [next_node])
```

In addition to directional path queries, we also implemented several functionalities that allow us to analyse the graph independently of path traversal. The filters are callable objects that can be applied to the entire graph.

For example, the `FilterByType` filter returns all nodes of a given type:

```
f_type = FilterByType("https://w3id.org/glass/ontology/WindowGlass")  
results = f_type.apply(graph)
```

Similarly, the `FilterByName` filter selects nodes whose label contains a given substring:

```
f_name = FilterByName("example.org")  
results = f_name.apply(graph)
```

Finally, the `FilterOrphans` filter identifies nodes with no incoming and no outgoing edges:

```
f_orphan = FilterOrphans()  
results = f_orphan.apply(graph)
```

Additionally, we implemented a small program that visualizes the graph using the `spring_layout`, which performs best on smaller graphs. We also developed a `shortest_path()` function based on breadth-first search to compute the minimum distance between any two nodes. Together, the methods and the filtering functionalities provide a flexible and expressive framework for analysing the graph, and our tests confirm that the results match those obtained in Protégé.

10 Task 10: Class Structure as E–R Diagram

In this task, we translated our class structure into an Entity–Relationship (E–R) diagram. We began by identifying the main classes in our system and treating each of them as an entity. After this, we considered how these classes interact, which led us to define the corresponding relationships. Then we proceeded by assigning values such as 1, 0..1, and 1.. N to represent the number of instances involved in each association. We also separated between partial and total participation by using single and double lines respectively. Through this made an E–R diagram that reflects the logical structure of the final product.

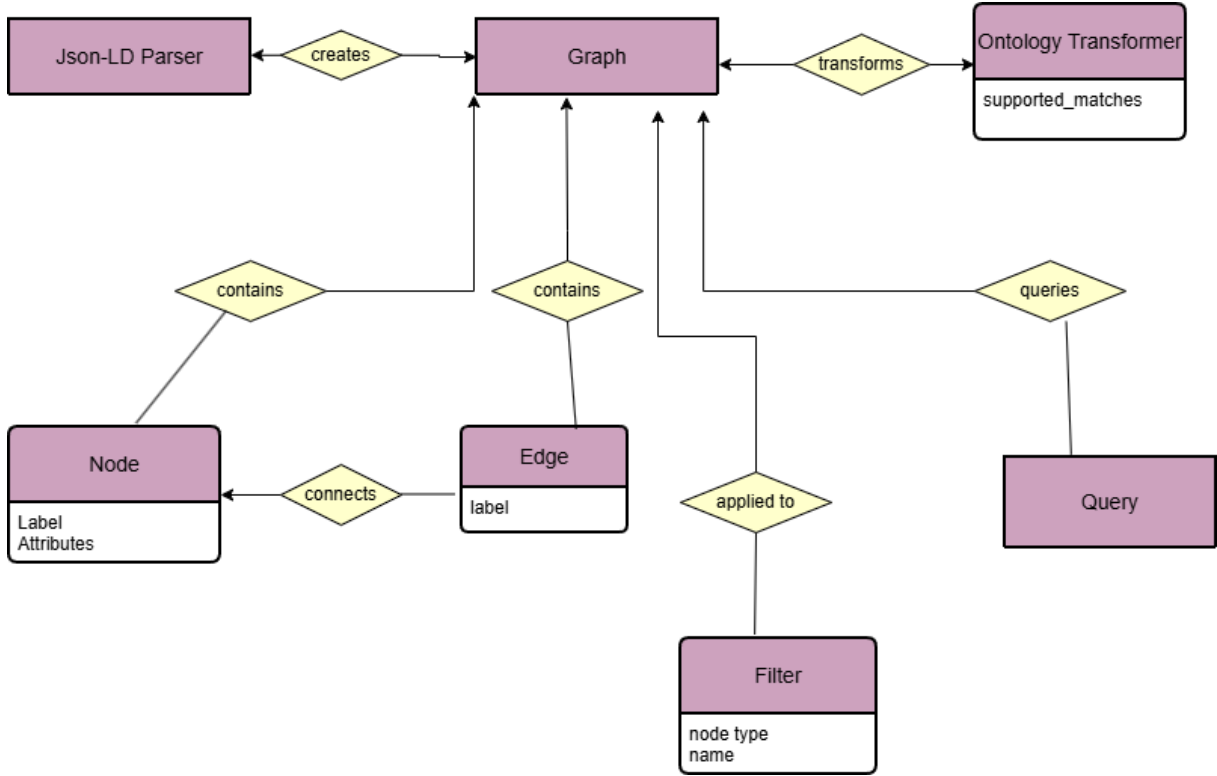


Figure 1: Entity Relationship Diagram

11 Task 11: Knowledge graph transformation implementation

The objective of this task was to implement a knowledge graph transformation mechanism. The goal was to create a new graph that follows a target ontology by using a source graph together with an alignment file describing correspondences between concepts and relations.

To solve this, we implemented a new class called `OntologyTransformer`. The class loads an alignment file and uses the mappings to transform labels found in the source graph. The implementation supports different mapping types such as `skos:exactMatch`, `skos:closeMatch`, `skos:broadMatch`, and `skos:narrowMatch`.

The transformation process starts by loading the source graph from a JSON-LD file. The alignment file is then read and converted into a dictionary containing mappings between source and target concepts. A new graph is created, and all nodes and edges from the source graph are copied into it. Whenever a label matches an entry in the alignment file, the corresponding target label is used instead. The original graph remains unchanged, while the transformed graph is stored separately.

To test the implementation, we created a test program that loads a source graph, loads the alignment file, performs the transformation, and exports the result as a new JSON-LD file. The

test program also prints information about the number of nodes and edges before and after transformation.

The output of the program is a transformed knowledge graph stored as a JSON-LD file. During testing, the source graph was loaded successfully, the alignment file was processed correctly, and a new graph was generated and exported. The transformed graph contained the same number of nodes and edges as the original graph, showing that the transformation process completed successfully and produced a valid output graph.

12 Task 12: Window Use Case

This task applies the CEON–DPPO–DPP alignment to a window-based Digital Product Passport (DPP) knowledge graph. We use the alignment to check which parts of the original graph (CEON + DPPO + Glass) can be converted into DPP terms. We only transform elements that have a valid SKOS mapping to DPP. Everything without a mapping stays as it is. Thus, the final graph is a **partial transformation**, not a full replacement.

12.1 Transformation Principle

- If a CEON or DPPO concept has a SKOS mapping to DPP, it is replaced.
- If no alignment exists, the original concept is preserved.
- Glass and IFC concepts are only indirectly affected if a mapping exists through DPPO.

12.2 Alignment Types Used

Mappings using `skos:exactMatch`

- **dppo:Product** `skos:exactMatch` **dpp:Product**
- **dppo:Substance** `skos:exactMatch` **dpp:Material**
- **dppo:DPP** `skos:exactMatch` **dpp:DPP**

Mappings using `skos:closeMatch`

- **dppo:CompositionInformation** `skos:closeMatch` **dpp:MaterialComposition**
- **dppo:ProductCharacteristic** `skos:closeMatch` **dpp:ProductCharacteristic**

Mappings using CEON → DPPO → DPP chain (via SKOS)

- **ceon:Item** → **dppo:Product** `skos:exactMatch` **dpp:Product**
- **ceon:MatterComposition** → **dppo:CompositionInformation** `skos:closeMatch` **dpp:MaterialComposition**

Concepts occurring in the data with *no* alignment

- **glass:Window**, **glass:WindowGlass**, **glass:WindowFrame** (remain unchanged)
- **ceon:compositionOf**, **ceon:associatedWithMatter** (remain unchanged)

12.3 Occurrence Analysis (CEON/DPPO to DPP Transformation)

The following table summarizes which elements in the source knowledge graph can be replaced based strictly on the provided CEON–DPPO and DPPO–DPP alignment files.

Source Concept (in data)	Alignment Available	Replaceable With (DPP)
ceon:Item	CEON $\rightarrow$ DPPO $\rightarrow$ DPP (via <code>skos:exactMatch</code>)	dpp:Product
ceon:MatterComposition	CEON $\rightarrow$ DPPO $\rightarrow$ DPP (via <code>skos:closeMatch</code>)	dpp:MaterialComposition
dppo:Product	DPPO $\rightarrow$ DPP (<code>skos:exactMatch</code>)	dpp:Product
dppo:Substance	DPPO $\rightarrow$ DPP (<code>skos:exactMatch</code>)	dpp:Material
dppo:CompositionInformation	DPPO $\rightarrow$ DPP (<code>skos:closeMatch</code>)	dpp:MaterialComposition
dppo:ProductCharacteristic	DPPO $\rightarrow$ DPP (<code>skos:closeMatch</code>)	dpp:ProductCharacteristic
dppo:DPP	DPPO $\rightarrow$ DPP (<code>skos:broadMatch</code>)	dpp:DPP
Concepts occurring in data but NOT replaceable		
glass:Window, glass:WindowGlass, glass:WindowFrame	No alignment in provided files	Not replaceable
ceon:compositionOf, ceon:associatedWithMatter	No alignment in provided files	Not replaceable

Table 1: Occurrences of CEON and DPPO in the use-case data that can be replaced with DPP concepts

12.4 Target Knowledge Graph (After Transformation)

After applying the alignment, only aligned entities are replaced with DPP terms.

```

ex:_a a dpp:Product .

ex:palletDPP_1 a dpp:DPP ;
  dpp:describes ex:_a ;
  dpp:containsInformation ex:_c , ex:_d , ex:_e , ex:_f .

ex:_c a dpp:MaterialComposition ;
  dpp:hasPart ex:_g , ex:_h , ex:_i .

ex:_f a dpp:Material ;
  dpp:compositionOf ex:_i ;
  dpp:associatedWithMatter ex:ALU .

ex:_e a dpp:ProductCharacteristic ;
  dpp:isAbout ex:_g ;
  dpp:value "2"^^xsd:integer .

```

12.5 Discussion

The transformation demonstrates partial ontology unification. Core product and material structures are successfully mapped to DPP, while CEON-specific relations such as `compositionOf` remain unchanged due to missing alignment. This confirms that alignment of the ontology does not guaranty full replacement.

13 Task 13: Testing

The objective of this task was to test and validate the implemented functionality of the system. Testing was carried out using four categories: unit tests, integration tests, regression tests, and acceptance tests.

The main focus of the test suite is unit testing, implemented using the `pytest` framework. These tests verify the behaviour of individual components in isolation. In addition, integration tests to validate interactions between major system components. Regression testing is represented by the roundtrip test. It guards against a specific bug where re-parsing an exported file produced an empty graph. The roundtrip test also serves as an acceptance test. It validates the system end to end: a file is parsed, transformed, exported, and re-parsed, confirming the full pipeline works as intended.

Table 2: Overview of implemented pytests.

Type	Component	Test goal
Unit	Node	Label storage on creation.
Unit	Node	Initialization of empty edge lists and attributes.
Unit	Node	Label property is settable after creation.
Unit	Node	Correct <code>__repr__()</code> output.
Unit	Node	Each instance has its own independent outgoing/incoming lists.
Unit	Node	Each instance has its own independent attributes dictionary.
Unit	Edge	Correct assignment of label on creation.
Unit	Edge	Correct assignment of source and target on creation.
Unit	Edge	Label property is settable after creation.
Unit	Edge	Source property is settable after creation.
Unit	Edge	Target property is settable after creation.
Unit	Edge	Correct <code>__repr__()</code> formatting.
Unit	Edge	Edge repr reflects updated source node label.
Unit	Graph	Graph initializes with no nodes and no edges.
Unit	Graph	<code>add_node</code> creates a node and stores it correctly.
Unit	Graph	Calling <code>add_node</code> twice with same label returns the same node.
Unit	Graph	<code>add_edge</code> returns an edge with correct label, source, and target.
Unit	Graph	<code>add_edge</code> automatically creates missing source and target nodes.
Unit	Graph	After <code>add_edge</code> , edge appears in source outgoing and target incoming.
Unit	Graph	<code>add_edge</code> appends the new edge to the graph edges list.
Unit	FilterByType	Returns True for a node with a matching type edge.
Unit	FilterByType	Returns False for a node with a non-matching type edge.
Unit	FilterByType	<code>apply()</code> returns only nodes matching the type.
Unit	FilterByName	Returns True when the search string appears in the node label.
Unit	FilterByName	Returns False when the search string is not found in the node label.
Unit	FilterByName	<code>apply()</code> returns only nodes whose label contains the substring.
Unit	FilterOrphans	A node with no edges is considered an orphan.
Unit	FilterOrphans	A connected node is not considered an orphan.
Unit	FilterOrphans	<code>apply()</code> returns only orphan nodes.
Unit	Query	Graph reference is stored correctly on creation.
Unit	Query	Forward traversal returns correct path.
Unit	Query	Forward traversal returns empty list when no path exists.
Unit	Query	Backward traversal returns correct path.
Unit	Query	Mixed-direction traversal returns empty list when no valid path exists.
Unit	Query	Multiple valid forward paths are all returned.
Unit	Query	Shortest path between two connected nodes.
Unit	Query	Shortest path returns single node when start equals end.
Unit	Query	Shortest path returns empty list when no connection exists.
Unit	Query	Returns one of the shortest routes when multiple exist.
Unit	JsonLD_parser	Root node receives correct type edge.
Unit	JsonLD_parser	Nested list objects produce one edge per item.
Unit	JsonLD_parser	Nested single object and its type edge are handled correctly.
Unit	JsonLD_parser	The same entity referenced twice is reused as one node.
Unit	JsonLD_parser	Correct edge count when duplicate removal is disabled.
Unit	JsonLD_parser	Context metadata is not included in the graph.
Unit	OntologyTransformer	Alignment mapping is loaded correctly from a JSON file.
Unit	OntologyTransformer	Multiple match types are accepted, not only <code>skos:exactMatch</code> .
Unit	OntologyTransformer	<code>_transform_label</code> returns the mapped label when one exists.
Unit	OntologyTransformer	<code>_transform_label</code> returns the original label when no mapping exists.
Unit	OntologyTransformer	<code>transform()</code> replaces mapped labels and leaves unmapped ones unchanged.
Unit	OntologyTransformer	<code>export_jsonld()</code> writes a correctly structured JSON-LD file.

Type	Component	Test goal
Unit	JsonLD-parser	Missing file raises <code>FileNotFoundError</code> .
Unit	JsonLD-parser	Malformed JSON raises <code>json.JSONDecodeError</code> .
Unit	OntologyTransformer	Missing alignment file raises <code>FileNotFoundError</code> .
Integration	Graph + Filter + Query	Filter results are consistent with forward query paths.
Integration	Graph + Query + Filter	Query and filter results agree on node classification.
Integration	JSON-LD + Graph + Query	Parsed graph supports expected forward query paths.
Integration	Graph + Ontology-Transformer + Query	Ontology transformation is reflected in query results.
Integration Regression Acceptance	JsonLD-parser + OntologyTransformer + JsonLD-parser	Full roundtrip: parse, transform, export, and re-parse produces identical node set. Guards against the <code>@graph</code> wrapper bug where re-parsing an exported file produced an empty graph.
Sanity	Test (addition)	Minimal addition assertion confirming the test framework itself is operational.

Table 3: Pytest files mapped to user stories.

Test file	User stories
test_Node.py	4.1, 4.3, 4.6
test_Edge.py	4.2, 4.6
test_Graph.py	4.1, 4.2, 4.3, 4.7, 2.6
test_Filter.py	1.4
test_query.py	1.3, 1.5, 1.6, 2.5
test_JsonLD_parser.py	1.2, 4.1, 4.2, 4.3, 2.6
test_OntologyTransformer.py	1.1, 1.2, 2.3
test_Error_wrong_file_etc.py	2.1
test_Integration_graph_filter_then_query.py	1.3, 1.4, 1.5
test_Integration_graph_query_then_filter.py	1.3, 1.4, 1.5
test_Integration_jsonld_graph_query.py	1.1, 1.2, 1.5
test_Integration_ontology_transformer_query.py	1.2, 1.5
test_integration_jsonld_roundtrip_through_transformer.py	1.1, 1.2, 2.3, 4.4
test_simpleadd.py	—

The test suite covers all core components at the unit level, with integration tests validating cross-component interaction.

Lastly, there was some additional testing done which doesn't quite fit in. Sanity tests (just to get used to pytest), Error-testing, and lastly some informal tests used during development. These didn't use the pytest package and were used for manual inspection and debugging: `check_W1.py`, `check_W2.py`, `check_W3.py`, `dumb-test.py`, and `nullNodeTest.py`.

In total, 60 pytest tests were implemented (not all "original"), as well as 5 informal scripts used for manual inspection and debugging during development.

The requirements register has been updated and shows what is tested (with one/more pytest), partially (no pytest), and no tested.

14 Task 14: Python Package

We prepared the final version of our code as a Python package. We organised the modules into a clear directory layout, added an `__init__.py` file to define the package, and made sure that functions and classes were properly documented.

15 Documentation of Generative AI Tools

We used a Generative AI tool during this project to help set up our LaTeX report, with the basic structure and formatting.

We also used it to better understand the concept of knowledge graphs and how incidence list graph structures work, in particular how nodes store incoming and outgoing edges. This helped us when implementing the graph in Python.

For Task 6, we also used it to better understand how Protégé can export knowledge graphs into JSON-LD format. This helped us correctly load the exported files into our Python implementation. We also used it to understand labels and predicates are represented in code. This helped us correctly match edge labels from the JSON-LD data when implementing path queries. We also used it to understand how SPARQL path queries map to sequences of edges in a graph, which helped us verify that our results matched Protégé. For task: 13: AI was used to map different types of test needed. After implementation AI was used to map the tests to the user stories, and find gaps in the implemented tests (user stories that were not tested).

16 Conclusion

The final system is packaged as a Python module that provides a knowledge graph implementation. It includes support for building graphs from JSON-LD data, querying paths in different directions, applying filters, some assorted functionalities, and transforming graphs based on ontology mappings.